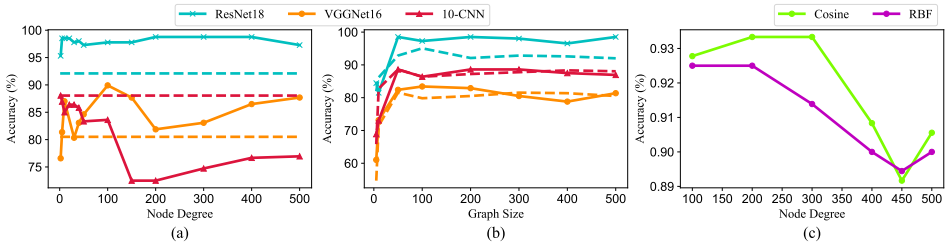
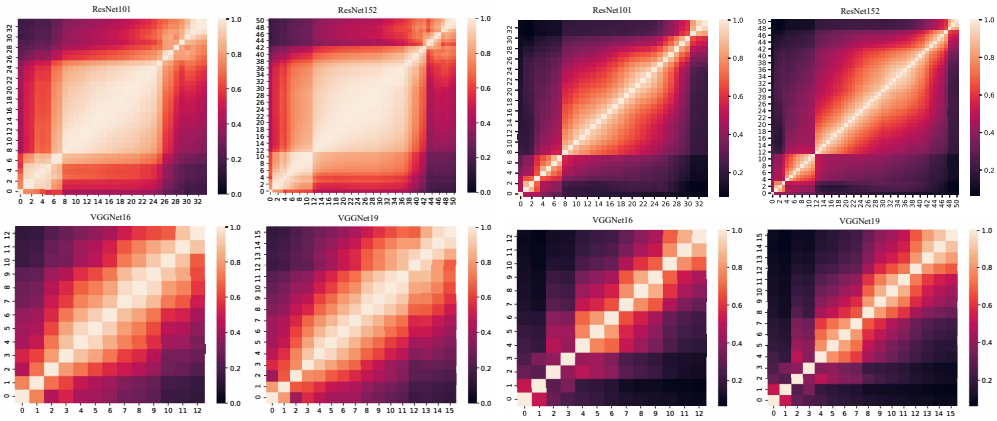


Figure 3: Sanity check accuracy using 10layer-CNN, VGGNet16, and ResNet18 on CIFAR10. In (a) and (b), the solid lines are the accuracy of GBS, while the dash lines are the accuracy of CKA; (c) shows the impact of kernels and sparsity, performed on 10layer-CNN.



(a) CKA (b) GBS

Figure 4: The similarity between the layers of VGGNets and ResNets on CIFAR10.

initialization utilizes Kaiming normalization [32] with ten different random seeds. The average accuracy of three groups of models is above 89%.

Given the intuition that an adequate number of nodes are beneficial to obtain a good similarity, we start with analyzing the influence of degree (top k connected nodes) under a fixed number of nodes (500). The 500 samples are randomly selected from the dataset that has a balanced label distribution. The result is shown in Fig. 3 (a), where the degree varies from 1 to 500 and we also calculated the unmodified linear CKA on 3 models. It shows that the degree of the generated graph has little impact on finding the layer correspondence. Even when the degree is set to 1, the accuracy only drops less than 6% compared with the degree set to 10. When the degree increase to 500, i.e., using a fully connected graph, the accuracy slightly drops, which indicates that a fully connected graph may bring redundant information and is detrimental to the extraction of relevant information. Overall, GBS has better performance in the sanity check. Considering the computational cost, we set the degree to 5, i.e., only connect the top 5 most correlated nodes in the graph.

Then we evaluate the performance of GBS with the graph size (number of nodes in the graph) varies from 5 to 500, as shown in Fig. 3 (b). Note that the number of categories of CIFAR10 is 10. When the graph size is set to 5, i.e., less than the number of categories of CIFAR10, GBS still achieves accuracy over 61.0% and 84.4% on VGGNet16 and ResNet18, respectively. When the graph size is greater than 50, the accuracy hardly changes with it.

We further show that the performance of CKA on sanity check can be improved by inducing sparsity and normalized edge weight in Fig. 3 (c). We use 500 input examples and 10layer-CNN. The Node Degree axis represents the reserved top k edges during CKA embeds representations into graphs, where 500 represents the unmodified CKA. It indicates that for RBF kernel and cosine kernel, sparsity can significantly improve the performance of CKA.

As shown in Fig. 4, we use the confusion matrix to demonstrate the similarity of different layers within a model. The models in Fig. 4 are trained on CIFAR10, we also provide the results of CIFAR100 and ImageNet, together with the detailed model structures in the appendix. For both VGGNets and ResNets, grids in the confusion matrices exactly correspond to the size and depth of the block in the model, i.e., the similarity of layers inside the same block is higher than that of layers in different blocks. GBS has a more clear and more fine-grained presentation for the models' architecture compared with CKA. Within a grid, the color gradually darkens with the diagonal as the center, which means inside the same block, the greater the distance between layers, the lower the similarity. Because within a block, DNN extracts features layer by layer, the similarity gradually decreases accordingly. The residual connections deliver the features of different depths between the entry of blocks, making the representations of the current block significantly different from the post block.

In summary, the ablation analysis indicates that GBS is robust to the choice of graph size and degree. The sanity check implies that GBS is a more fine-grained similarity index than CKA. Increasing degree values slightly helps improve the performance of GBS. Considering the computational cost, an excessively large number of nodes and large degree values are unnecessary.

4.3 Functional Similarity and Motifs

We show that motif ratio can measure the functional similarity by two experiments, 1) stitching trained and randomly initialized models; and 2) stitching models trained by different amounts of data and epochs. ResNet18 and CIFAR10 are adopted in this experiment. All stitching layers are trained with 100 epochs, using SGD optimizer, and the learning rate is set to 0.1.

Figure 5: Variation of motif with the (a) stitch layer, (b) training data, and (c) training epochs.

As shown in Fig. 5, we use color to distinguish accuracy and motif ratio and use different line styles to distinguish stitch cases. Red lines denote the accuracy after stitching and blue lines denote the motif ratio after stitching. Solid lines with circle denote stitching two models of different initialization, dash lines with rectangle denote stitching a trained model with a random bottom network, and dot lines with cross denote stitching a random top model with a trained model.

The functional similarity depends not only on whether the extracted features of the bottom model are effective but also on the strength of the top model that processes the features. In Fig. 5 (a), when the top and bottom model matches (both trained), the motif ratio is high as they are highly functionally similar. For a random bottom model, since it can not provide useful representation, the motif, as well as functional similarity, is low at all the layers. A trained bottom model can provide informative representation for classification, nevertheless, the top model is a random model with a poor ability to process these representations, thus the motif, i.e., functional similarity increases with the layer depth.

Previous works [6] have shown that training with more data and epochs provides functionally more similar representations. We stitch a model trained with different samples and epochs to another fully trained model, to show how the motif varies with the amount of data and training epochs. As shown in Fig. 5 (b) and Fig. 5 (c), two models are stitched in the second block. The motif ratio is positively correlated with functional similarity, which indicates that the motif ratio is closely related to the functional similarity. Features with weak functionality decrease connections between same category samples, while functionally stronger features bring more dense connections.

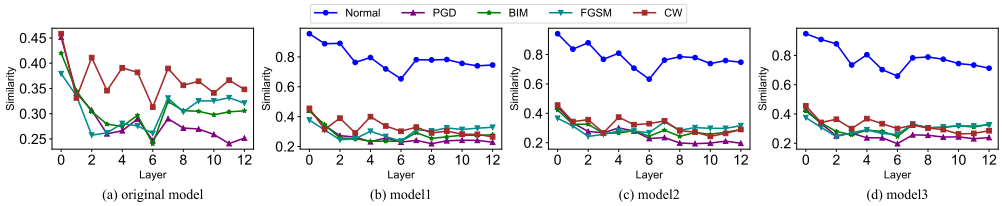
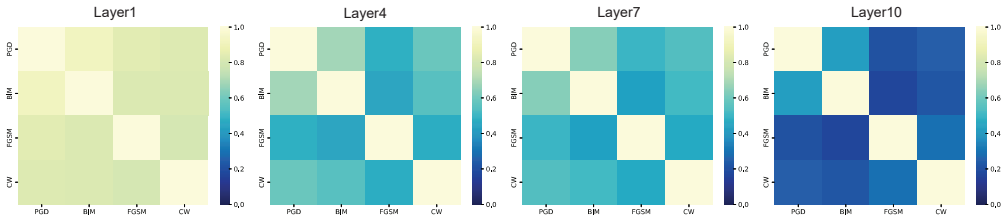
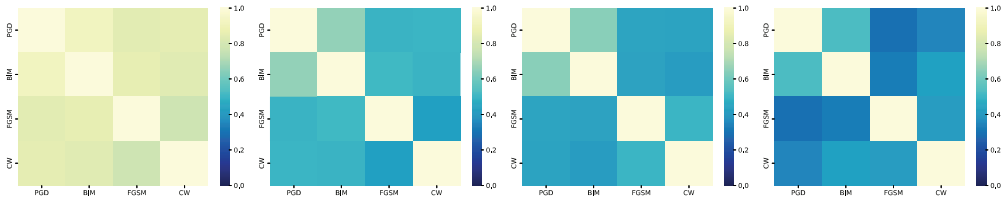


Figure 6: The representational similarity of adversarial samples transferring to different models. In (a), the similarity is calculated between normal and adversarial samples. In (b), (c), and (d), we also calculate the similarity between original model and other models using normal samples.



(a) Targeted Attacks.



(b) Untargeted Attacks.

Figure 7: The similarity of different adversarial attacks.

Overall, our explicitly constructed graph bridges the gap between functional similarity and feature similarity. The similarity between graphs manifests the feature similarity while the graph motif is correlated with the functional similarity.

4.4 Representation of Adversarial Samples

In the end, we use GBS to reveal the transferability of adversarial attacks in representations and the difference between targeted and untargeted attacks. To analyze whether the internal representations are similar when the attack transfers to different models, we generate adversarial samples against one model and attack other models with them. The models used here are VGGNets of different initialization. We use *advertorch* [19] package to generate untargeted adversarial samples for the VGGNet16 model trained on CIFAR10. The adversarial attacks include Fast Gradient Sign Method [27] (FGSM), Carlini & Wagner Attack [11] (CW), Basic Iterative Method [40] (BIM), and Project Gradient Descent [43] (PGD). All the attacks use the default parameters and the attack success rates of PGD, BIM, FGSM, and CW for the original model are 56%, 50%, 41%, and 54%, respectively. The batch size is set to 100. As shown in Fig. 6 (a), compared with the normal samples, all attacks show that the similarity decreases with the increase of depth. When the same batch of normal samples are fed into other models, as shown in Fig. 6, the representations show relatively high similarity at all depths. However, when fed with the same batch of adversarial samples, the adversarial samples successfully fool them with different representations activated in the other three models. It indicates that the transferability of adversarial samples is not because the same features are activated.

In Fig. 7, we show the similarity between different types of adversarial examples in the hidden layer space. We calculate the similarity between different adversarial attacks every 3 layers in VGGNet16. Targeted attacks and untargeted attacks are calculated separately and the target label is all set to the same one. For both kinds of attacks, shallow layers have a high similarity between different attacks,